EffortGraph: Joint Model and Reasoning Effort Selection with Graph Neural Networks

Chenyue Li
Stanford University
chenyuel@stanford.edu

Yanzhen Shen
Stanford University
yanzhen4@stanford.edu

1 Progress and research question

Choosing how much computation to spend on a query requires deciding both which LLM to use and how much reasoning it should perform. Our proposal asks whether shared task, model, and effort structure can improve this decision through a heterogeneous graph. At this milestone, we have completed response collection, benchmark grading, and non-graph routing experiments. We first test whether label rebalancing and a larger trainable text encoder improve routing before introducing message passing. The graph router remains the next stage of the project.

Our current results show that simply increasing the number of trainable router parameters does not yield consistent improvements in quality–cost reward. Warm-started encoder fine-tuning recovers some of the cost-aware performance lost through rebalancing, but does not outperform the original frozen-encoder MLP on those objectives. Code, splits, held-out predictions, and reproducible result tables are available at `https://github.com/yanzhen4/cs224w_effortgraph`.

2 Data collection and evaluation protocol

Benchmarks. We collected a pilot of 800 questions across 15 dataset-specific task groups (Table 1), using MMLU-Pro [2], SuperGPQA [3], and LiveBench [4]. SuperGPQA contributes 20 easy, 20 middle, and 20 hard questions per field. LiveBench sampling covers subtasks and available native difficulty labels. We retain 127 historically removed LiveBench questions in this pilot. Questions are deduplicated within the sampling pools.

Table 1: Collected pilot. Each fold holds out one fifth of each task group.

Dataset	Task groups	Questions
MMLU-Pro	Math, physics, health, law, history	250
SuperGPQA	Physics, mathematics, chemistry, clinical medicine, biology	300
LiveBench	Mathematics, reasoning, coding, data analysis, instruction following	250

Configurations and grading. Each question is evaluated with `gpt-5-nano`, `gpt-5-mini`, and `gpt-5` (2025-08-07 snapshots), each at low, medium, and high reasoning effort. This gives 7,200 recorded responses. We use zero-shot prompts and a 64,000-token output limit including reasoning. Multiple-choice responses are scored by extracted-answer exact match; LiveBench uses task-specific grading, including code execution and an equivalence-judge fallback for AMPS mathematics. Three incomplete responses receive zero credit. Quality is the benchmark score, including partial credit where supported. Cost is calculated from recorded input, cached-input, and output tokens at the recorded list prices; reasoning tokens are included in output cost.

Changes from the proposal. We reduced the proposed 1,800-question collection to an 800-question pilot and used GPT-5 configurations instead of GPT-5.4 to accommodate endpoint throughput. We replaced the single train/test split with five-fold cross-validation and retained historical LiveBench questions. Public benchmark exposure remains a limitation. The current aggregate is a mean over questions, rather than the dataset-macro average proposed for the final evaluation.

Splits and reward. Each fold contains 640 training and 160 test questions. All nine responses to a question remain in the same fold. For configuration $a = (m, e)$, we use

$$r(q, a) = \alpha\widetilde{p}(q, a) - \beta\widetilde{c}(q, a), \qquad y_{q,a} = \mathbf{1}\Big[a = \arg\max_{a'} r(q, a')\Big]. \tag{1}$$

Here $\widetilde{p}$ is min–max normalized benchmark quality and $\widetilde{c}$ is min–max normalized log dollar cost. Normalization bounds are fitted on the original training fold and reused for every method. Ties are resolved by lower dollar cost, then a fixed configuration order. We train separate routers for quality-first $(\alpha, \beta) = (1, 0)$, balanced $(0.5, 0.5)$, and cost-first $(0.2, 0.8)$ objectives. Every reported test decision is out of fold.

3 Routing experiments

Frozen-encoder MLP. We encode query text, including answer choices, and task descriptions with `all-MiniLM-L6-v2`. The encoder uses at most 256 tokens and produces normalized 384-dimensional mean-pooled representations. Separate $384 \to 32 \to 32$ MLPs project query and task features. Learned model and effort embeddings each have dimension 32. Their concatenation is scored by a $128 \to 32 \to 1$ MLP for each of the nine configurations. This router has 31,105 trainable parameters and no graph message passing. We train with binary cross-entropy against the one-hot labels, Adam, learning rate 10^{-3}, batch size 32, and 1,000 epochs, retaining the final checkpoint.

Training-label rebalancing. The cheapest configuration, nano low, accounts for 67.75%, 67.75%, and 77.66% of training labels across the three objectives. We construct a separate *Balance* training subset for each fold and objective by removing only nano-low-labeled examples. Each training fold shrinks from 640 to 320 questions, preserving the original dataset ratio: 100 LiveBench, 120 SuperGPQA, and 100 MMLU-Pro questions. Task quotas are approximately preserved within each dataset. Nano-low label shares fall to 35.50%, 35.50%, and 55.31%, respectively. Gold labels and normalization remain fixed, and the 160 test questions are unchanged. This experiment changes the training-label distribution; it does not rebalance the evaluation set.

Joint encoder and head training. On the Balance subsets, we compare two initializations. *Warm start* initializes the routing head from the corresponding trained Balance MLP and then updates the encoder and head jointly. *Fresh head* initializes the head randomly and also updates both jointly; its encoder still starts from pretrained MiniLM. Unfreezing the encoder increases the trainable parameter count to 22,596,481, approximately $726\times$ the frozen-encoder baseline. We use AdamW with encoder learning rate 2×10^{-5}, head learning rate 10^{-4}, weight decay 0.01, gradient clipping at 1, and effective batch size 32 (microbatch 8, accumulation 4). Both variants train for 50 epochs, saving checkpoints every 10 epochs and evaluating the final checkpoint. We use no validation-based checkpoint selection.

Comparison scope. Frozen-encoder experiments cover five seeds; full fine-tuning currently covers seed 0 across all five folds and three objectives. Table 2 therefore compares seed 0 for every learned method, aggregating 800 held-out decisions per objective. The runs share test questions but are not compute-matched: warm start includes the preceding MLP training. The repository also reports the multi-seed frozen-encoder results.

4 Results and interpretation

Rebalancing trades cost for quality. Balance MLP improves quality-first reward from 0.6741 to 0.6923, but lowers balanced reward from 0.1835 to 0.1708 and cost-first reward from -0.0865 to -0.1266. Reducing the frequency of cheap configurations in the training labels is therefore not sufficient to improve routing utility on the original test distribution.

Table 2: Mean held-out reward, seed 0, five folds; higher is better. Bold marks the best non-oracle result in each column. The oracle uses observed outcomes and is a retrospective reference, not a deployable router.

Strategy	Quality-first	Balanced	Cost-first
GPT-5-nano low	0.6005	**0.1879**	**-0.0597**
GPT-5-mini low	0.7292	0.1816	-0.1470
GPT-5 high	**0.7942**	0.0606	-0.3795
Original MLP	0.6741	0.1835	-0.0865
Balance MLP	0.6923	0.1708	-0.1266
Full fine-tuning, warm start	0.6916	0.1801	-0.1088
Full fine-tuning, fresh head	0.6005	**0.1879**	**-0.0597**
Quality-first oracle	0.8975	0.3055	-0.0498

Additional trainable capacity does not give a consistent gain. Warm-started fine-tuning leaves quality-first performance essentially unchanged relative to Balance MLP (0.6923 versus 0.6916). It improves balanced and cost-first rewards to 0.1801 and -0.1088, but both remain below Original MLP. For the balanced objective, its mean quality rises from 0.6904 to 0.7088 while mean generation cost falls from \$0.002087 to \$0.001981 per query. These local improvements do not translate into an advantage over the original training setup. With a fresh head, the jointly trained model selects nano low for every held-out query under all three objectives, reproducing a fixed policy rather than learning useful query-dependent routing.

Fixed policies remain strong. None of the learned routers exceeds the best fixed configuration for its objective among the nine candidates. GPT-5 high provides the highest fixed-policy quality, while nano low wins both cost-aware objectives. The quality-first oracle selects the highest-scoring response for each question, breaking ties by lowest cost; when no configuration earns full credit, it retains the best available score. It reaches quality 0.8975 at \$0.002346 per query. This indicates room for outcome-informed selection, although this oracle does not maximize each cost-aware reward separately.

5 Next steps

The current evidence suggests that increasing trainable parameters alone does not resolve this routing problem. This is a finding for the tested optimization and data settings, rather than a statistical claim across seeds. We next implement the proposed task–query–model–effort graph and compare it against these baselines on the same held-out questions. Following GraphRouter [1], we will test whether shared response structure provides information beyond text features and learned identifiers. We will mask the target query's response edges and exclude all test outcomes from message passing. Effort-node and task-node ablations, additional seeds, and per-dataset quality–cost analysis will distinguish structural gains from optimization effects. Feedback on the supervision objective and the treatment of cheap-configuration dominance would be particularly useful.

References

[1] Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A Graph-based Router for LLM Selections. ICLR, 2025. https://arxiv.org/abs/2410.03834.

[2] Yubo Wang et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. NeurIPS Datasets and Benchmarks, 2024. https://arxiv.org/abs/2406.01574.

[3] SuperGPQA Team. SuperGPQA: Scaling LLM Evaluation across 285 Graduate Disciplines. 2025. https://arxiv.org/abs/2502.14739.

[4] Colin White et al. LiveBench: A Challenging, Contamination-Free LLM Benchmark. ICLR, 2025. https://arxiv.org/abs/2406.19314.